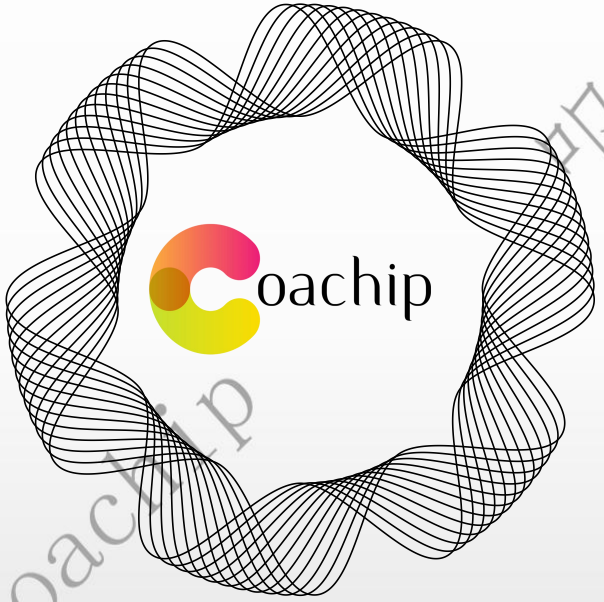



# SV的随机约束控制

叩特电子Coachip

# 课程内容及目标

## 课程内容：

- 介绍控制约束的关键字constraint_mode
- 介绍控制变量约束的关键字rand_mode
- 介绍控制变量约束的关键字randomize() with {}
- 介绍randomize单独控制变量
- 介绍pre_randomize 与 post_randomize函数




叩特电子Coachip

# 控制多个约束块 constraint_mode()

```
class Packet;
  rand int length;
  constraint c_short {length inside {[1:32]};}
  constraint c_long  {length inside {[1000:1023]};}
endclass
Packet p;
initial begin
  p = new();
  p.c_short.constraint_mode(0); //disabling short constraint
  assert(p.randomize());

  p.constraint_mode(0);         //disabling all constraints
  p.c_short.constraint_mode(1); //enabling short constraints
  assert(p.randomize());
  transmit(p);
end
```

叩特电子Coachip

# 控制随机变量 rand_mode()

```
class Packet;
  rand bit [7:0] length, payload[];
  constraint c_valid {length > 0;payload.size() == length;}
endclass

Packet p;
initial begin
  p = new();
  p.length.rand_mode(0);   // Make length nonrandom
  p.length = 42;           // set it to a constant value
  assert (p.randomize()); // then randomize the payload
  p.length.rand_mode(1);   // Make length random
end
```

叩特电子Coachip

# 控制随机变量 randomize() with {}

```
class Transaction;
  rand bit [31:0] addr, data;
  constraint c1 {addr inside{[0:100],[1000:2000]};}
endclass

Transaction t;
initial begin
  t = new();
  // addr is 50-100, 1000-1500, data < 10
  assert(t.randomize() with {addr >= 50; addr <= 1500; data < 10;});
  $display("t::addr=%d,data=%d",t.addr,t.data);
  // force addr to a specific value, data > 10
  assert(t.randomize() with {addr == 2000; data > 10;});
  $display("t::addr=%d,data=%d",t.addr,t.data);
end
```

叩特电子Coachip

# 控制随机变量 randomize单独控制变量

```
class Rising;
  byte low; // Not random
  rand byte med, hi; // Random variable
  constraint c_up {low < med; med < hi;}
endclass

Rising r;
initial begin
  r = new();
  r.randomize();     // Randomize med, hi; low untouched
  r.randomize(med); // Randomize only med; hi retains its previous value
  r.randomize(low); // Randomize only low(nonrandom variable).
end
```

即将电子Coachip

# pre_randomize与post_randomize函数

他们都是function, 不消耗仿真时间;

pre_randomize: 在执行randomize() 之前调用.
主要用来设定与randomize相关的各个变量的值；

post_randomize: 在randomize()之后调用.
主要用来调用相关的函数对randomize出来的变量进行后续处理。

他们通常用来设置一些非随机的变量或者计算随机数据中的错误

如randomize调用失败，那么post_randomize将不会被执行，但是pre_randomize不管randomize是否成功都会执行

叩特电子Coachip

# pre_randomize与post_randomize函数

```
class wr_tran;
  int constraint_en;
  int broadcast;
  rand int wr_addr;
  rand int wr_data;
  rand bit valid;
  constraint  generic_c { valid ==1; wr_addr<100;}
  function void pre_randomize();
     $display("call the pre_randomize !");
     if(!constraint_en) generic_c.constraint_mode(0);
  endfunction
  function void post_randomize();
     $display("call post_randomize !");
     if(wr_addr ==1) broadcast =1;
     else broadcast =0;
  endfunction
endclass
```

```
module m1;
  wr_trans m_tr;
  initial begin
    m_tr =new();
    m_tr.constraint_en=0;
    m_tr.randomize() with {wr_addr==200;
                           wr_data==3;
                           valid==0;};
    $display("wr_addr=%d,wr_data=%d,
              valid=%d,broadcast=%d",
              m_tr.wr_addr, m_tr.wr_addr,
              m_tr.valid,m_tr.broadcast);
  end
endmodule
```

叩特电子Coachip

# 约束的重载

场景一：采用constraint_mode(0)关闭约束，然后用randomize() with {} 重新约束；

```
class constrainend;
  rand bit x;
  rand bit [1:0] y;
  constraint c_xy {(x==0)-> y==0};
endclass
```

```
program test;
  constrainend p;
  initial begin
    p=new();
    p.c_xy.constraint_mode(0); //disable c_xy constraint
    assert(p.randomize() with {x==0; y==0;});
    $display("x = %d",p.x);
    $display("y = %d",p.y);
   end
endprogram
```

叩特电子Coachip

# 约束的重载

场景二：子类extends父类，然后定义同名约束覆盖父类约束；

```
class constrainend;
  rand bit x;
  rand bit [1:0] y;
  constraint c_xy {(x==0)-> y==0;}
endclass

class overwrite extends constrainend;
  constraint c_xy {x==0; y==0;}
endclass
```

```
program test;
 overwrite p;
  initial begin
    p=new();
    assert(p.randomize());
    $display("x = %d",p.x);
    $display("y = %d",p.y);
  end
endprogram
```

叩特电子Coachip

# 练习一：使用constraint_mode()

```
class Packet;
  rand int length;
  constraint c_short {length inside {[1:32]};}
  constraint c_long  {length inside {[1000:1023]};}
endclass
Packet p;
initial begin
  p = new();
  p.c_short.constraint_mode(0);
  assert (p.randomize());
  $display("p::length is %d", p.length);//做练习之前，先分析一下，这个值应该是多少？
  p.constraint_mode(0);
  p.c_short.constraint_mode(1);
  assert (p.randomize());
  $display("p::length is %d", p.length);
end  //思考：是否可以全部将c_short和c_long打开？
```

叩特电子Coachip

# 练习二：使用rand_mode()

```
class Packet;
  rand bit [7:0] length, payload[];
  constraint c_valid {length > 0;payload.size() == length;}
endclass
Packet p;
initial begin
  p = new();
  p.length.rand_mode(0);
  p.length = 42;
  assert (p.randomize());
  $display("p::length is %d", p.length); //思考：为什么p.length=42，而不是被随机化？
  p.length.rand_mode(1);
  assert (p.randomize());
  $display("p::length is %d", p.length); //思考：为什么p.length又被随机化了？
end
```

叩特电子Coachip

# 练习三：使用randomize() with {}

```
class Transaction;
  rand bit [31:0] addr, data;
  constraint c1 {addr inside{[0:100],[1000:2000]};}
endclass

Transaction t;
initial begin
  t = new();
  assert(t.randomize() with {addr >= 50; addr <= 1500; data < 10;});
  $display("t::addr is %d", t.addr);
  $display("t::data is %d", t.data);
  assert(t.randomize() with {addr == 2000; data > 10;});
  $display("t::addr is %d", t.addr);
  $display("t::data is %d", t.data);
end
```

思考：with{}当中的约束和class中的约束是什么关系？

叩特电子Coachip

# 练习四：使用randomize单独控制变量

```
class Rising;
  byte low;
  rand byte med, hi;
  constraint c_up {low < med; med < hi;}
endclass
Rising r;
initial begin
  r = new();
  r.randomize();
  $display("r::low is %d,r::med is %d,r::hi is %d",r.low,r.med,r.hi);
  r.randomize(med);
  $display("r::low is %d,r::med is %d,r::hi is %d",r.low,r.med,r.hi);
  r.randomize(low);
  $display("r::low is %d,r::med is %d,r::hi is %d",r.low,r.med,r.hi);
end
```

思考：为什么打印的hi数值一致？

叩特电子Coachip

# Q&A

昂特电子Coachip